

ELITE EXHAUST CARE – LPG LEAKAGE AND SMOKE DETECTION SAFETY SYSTEM

Project Documentation

Nobel Software Solutions

1. INTRODUCTION

Kitchen safety is a major concern in both residential and commercial environments due to the widespread use of LPG for cooking. LPG is highly flammable, and even a small leakage can result in serious accidents if not detected early. Smoke generation due to overheating, burning food, or electrical faults can also lead to fire hazards and health problems. Traditional safety methods such as manual inspection or standalone gas detectors are often insufficient, especially when users are not present in the kitchen.

In many cases, gas leakage occurs during night hours or when the kitchen is unattended, leading to delayed response and severe consequences. Conventional gas detectors usually trigger only a local alarm, which may go unnoticed. There is a need for an intelligent system that not only detects hazardous conditions but also informs users immediately and takes automatic safety actions.

The **Elite Exhaust Care – LPG & Smoke Detection and Safety System** is designed to address these challenges by combining sensor technology with IoT-based communication. The system continuously monitors LPG gas levels and smoke concentration using dedicated sensors. The collected data is analysed by a controller unit that determines whether the conditions are safe or hazardous.

When unsafe conditions are detected, the system immediately activates alert mechanisms such as buzzers or exhaust fans. Simultaneously, notifications are sent to the user's mobile device through an IoT platform. This ensures real-time awareness and faster response, even when users are away from home.

The proposed system enhances kitchen safety by minimizing human dependency and enabling automated response. It is scalable and adaptable for different environments such as homes, restaurants, and hotels. By integrating automation and remote monitoring, Elite Exhaust represents a modern and reliable approach to preventing gas leakage and smoke-related accidents.

1.1 Organization Profile

NOBEL SOFTWARE SOLUTIONS is a professional website designing, Customized software development, Business process Outsourcing – IT/ITES & Internet marketing company providing full featured web service including B2A Acquisition & B2C e-commerce solutions and acting as an offshore development centre for overseas development firms. NOBEL SOFTWARE SOLUTIONS is an innovative company, based in India that provides a series of Web-based and software and software applications that have helped their customer to create successful business ventures through online initiatives. NOBEL SOFTWARE SOLUTIONS provide all the services that a company needs to get online from web designing to web hosting and manage leading-edge web sites and e-business software solutions success in the domestic market. Ever since its inception, NOBEL software solutions have accrued continuous growth in all its business functions and this has been possible only due to its commitment, quality training methodologies, the services it offers, knowledge sharing with industry leaders and professional approach.

Services

- Professional Web Design
- SEO concerts, Internet Marketing
- Pay per click Campaign.
- Link Building, Ecommerce Solution
- Web Application Development
- Multimedia Presentations
- Customized Software development
- Business Process Outsourcing-IT/IT.

1.2. SYSTEM SPECIFICATIONS

1.2.1. Hardware Requirements

For IoT Device Setup:

- Microcontroller: ESP8266 (NodeMCU)
- Sensor: MQ-2 / MQ-6 Gas Sensor
- Power Supply: 5V DC Adapter / USB Power
- Connectivity: Wi-Fi Network
- Additional Components: Jumper Wires, Breadboard / PCB

For Monitoring System:

- Processor: Intel Dual Core or above
- Processor Speed: 2.0 GHz or higher
- RAM: 4 GB RAM (Minimum 2 GB)
- Hard Disk: 250 GB or above
- Monitor: 15-inch Colour Monitor or higher
- Input Devices: Standard Keyboard and Scroll Mouse
- Smartphone: Android / iOS device with internet access (for Blynk App monitoring)

1.2.2. SOFTWARE SPECIFICATION

- Programming Language: Micro python
- IoT Platform: Blynk IoT Platform
- Microcontroller IDE: Arduino IDE
- Operating System: Windows 10 or above
- Mobile Application: Blynk App (Android / iOS)
- Browser (Optional): Google Chrome / Edge for Blynk Web Dashboard

1.3. SOFTWARE DESCRIPTION

INTRODUCTION TO FRONT END

PYTHON

Python is a high-level, interpreted and general-purpose dynamic programming language that focuses on code readability. The syntax in Python helps the programmers to do coding in fewer steps as compared to Java or C. Python is widely used in bigger organizations because of its multiple programming paradigms. They usually involve imperative and object-oriented functional programming. It has a comprehensive and large standard library that has automatic memory management and dynamic features.

The software development companies prefer Python language because of its versatile features and fewer programming codes. Nearly 14% of the programmers use it on the operating systems like UNIX, Linux, Windows and Mac OS. Interactive

Advantages or Benefits of Python

The Python language has diversified application in the software development companies such as in gaming, web frameworks and applications, language development, prototyping, graphic design applications, etc. This provides the language a higher plethora over other programming languages used in the industry. Some of its advantages are-

Extensive Support Libraries

It provides large standard libraries that include the areas like string operations, Internet, web service tools, operating system interfaces and protocols. Most of the highly used programming tasks are already scripted into it that limits the length of the codes to be written in Python.

Presence of Third-Party Modules

The Python Package Index (PyPI) contains numerous third-party modules that make Python capable of interacting with most of the other languages and platform

Integration Feature

Python integrates Enterprise Application Integration that makes it easy to develop Web services by invoking COM or COBRA components. It has powerful control capabilities as it calls directly through C, C++, or Java via Jython. Python also processes XML and other markup languages as it can run on all modern operating systems through same byte code

Improved Programmer's Productivity

The language has extensive support libraries and clean object-oriented designs that increase two to tenfold of programmer's productivity while using the languages like Java, VB, Perl, C, C++, and C#.

Productivity

With its strong process integration features, unit testing framework and enhanced control capabilities contribute towards the increased speed for most applications and productivity of applications. It is a great option for building scalable multi-protocol network applications.

Open Source and Community Development

Python language is developed under an OSI-approved open-source license, which makes it free to use and distribute, including for commercial purposes.

Further, its development is driven by the community which collaborates for its code through hosting conferences and mailing lists and provides for its numerous modules.

Learning Ease and Support Available

Python offers excellent readability and uncluttered simple-to-learn syntax which helps beginners to utilize this programming language. The code style guidelines, PEP 8, provide a set of rules to facilitate the formatting of code. Additionally, the wide base of users and active developers has resulted in a rich internet resource bank to encourage development and the continued adoption of the language.

User-friendly Data Structures

Python has built-in list and dictionary data structures which can be used to construct fast runtime data structures. Further, Python also provides the option of dynamic high-level data typing which reduces the length of support code that is needed.

Simple and easy to Learn

1. The syntaxes of python language are very simple. Anybody can remember the python language syntaxes, rules and regulations very easily.
2. The elegant syntaxes of the python language make the people to learn python in easiest manner.
3. With out having any other programming languages knowledge, we can learn python directly.
4. The simple and powerful syntax of python language makes the programmers to express their business logic is less lines of code.

Platform Independent

1. Like Java programs, Python programs are also platform independent.
2. Once we write a Python program, it can run on any platform without rewriting once again.
3. Python uses PVM to convert python code to machine understandable code.

High Level Language:

1. Python is a high-level language.
2. While developing python applications, developers no need to bother about memory management.

Dynamically Typed Language:

1. Python is a dynamically typed language.
2. I mean, in python there is no need to declare type of variable.
3. Whenever I assign a value to the variable, based on the value the type will be allocated automatically.

Python supports POP & OOP

1. Python language supports both Procedural Oriented Programming and Object-Oriented Programming features.
2. I can implement OOPs features like Encapsulation, polymorphic, inheritance and abstraction in python programming language.

Python is Interpreted Language

1. Like PHP, Python is a Interpreted language.
2. Python applications do not require explicit compilation so that compiler is not require for python software.
3. Python interpreter is responsible for execution of python applications.
4. Whenever I run a python application, the python interpreter checks for the syntax errors. If there is no syntax errors found in our code, then the interpreter converts the code into intermediate code in the form of low-level format and executes it.
5. The intermediate code of the python applications is known as byte code.
6. The extension of the byte code file is. pye (Python Compile Code

Python is Embeddable

1. I can embed the python code into other languages such as C, C++, Java Etc.
2. To provide the scripting capabilities to other languages we can use the python code in it.

Develop GUI & Web Application

1. I can develop the GUI based applications using Python Languages.
2. I can also develop the Web applications using Python Language.

Applications of Python:

- GUI based desktop applications
 - Image processing and graphic design applications
 - Scientific and computational applications
 - Games
- Web frameworks and web applications
- Enterprise and business applications
- Operating systems
- Language development
- Prototyping

2. SYSTEM STUDY

2.1. EXISTING SYSTEM

Existing kitchen safety systems mainly rely on standalone LPG gas detectors or smoke alarms. These devices typically generate a local alarm when gas leakage or smoke is detected. However, they do not provide remote notifications or automated safety actions. In many households, no detection system is installed at all, increasing the risk of accidents. Manual monitoring is unreliable and unsafe, especially when the kitchen is unattended. Lack of integration with modern communication systems makes traditional solutions less effective in preventing major accidents.

2.1.1. DRAWBACKS

- LPG leakage is mainly detected by human smell perception.
- Some kitchens use standalone gas leak alarms with buzzer only.
- No automatic gas exhaust mechanism in most existing systems.
- No remote alert or notification system.
- Delayed response increases risk of fire or explosion.
- Existing systems do not support IoT-based monitoring.

2.2. PROPOSED SYSTEM

The proposed **Elite Exhaust** system uses LPG and smoke sensors integrated with IoT technology to continuously monitor kitchen safety. When gas leakage or smoke is detected beyond safe limits, the controller activates alarms and safety devices automatically. At the same time, alerts are sent to users through mobile or cloud applications. The system enables real-time monitoring and quick response, reducing the risk of fire accidents. It ensures enhanced safety by combining detection, automation, and remote accessibility.

2.2.1. ADVANTAGES

- Gas Leakage Detection
- Uses gas sensor to detect LPG concentration in air.
- Automatic Alarm System
- Buzzer alerts nearby people immediately.
- Automatic Exhaust Control
- Relay activates exhaust fan to remove accumulated LPG.
- IoT-Based Notification
- Node MCU sends email alerts to the user.
- Real-Time Monitoring
- Continuous monitoring of LPG levels.
- Low Cost & Energy Efficient
- Suitable for home and commercial kitchens.

3. SYSTEM DESIGN AND DEVELOPMENT

SYSTEM DESIGN

3.1 File Design

The file design of the system organizes all project resources into a well-structured directory hierarchy to ensure easy maintenance and scalability. The main application file (app.py) controls routing, request handling, and interaction with the Stable Diffusion model. Template files stored inside the templates folder define the user interface layouts, while static resources such as CSS stylesheets, uploaded images, and generated results are stored inside the static directory. Model-related files such as the Stable Diffusion. safe tensors file and CLIP model files are placed inside a separate model's folder to clearly separate application logic from large model data.

The system also includes a local SQLite database file (database.db) that stores generation metadata such as prompts, settings, and file paths of output images. Uploaded images are saved inside static/uploads, and generated images are saved inside static/results. This file organization improves access speed, simplifies debugging, and ensures that different components of the system do not interfere with each other.

3.2 Input Design

The input design focuses on collecting user data in a structured and user-friendly manner through HTML forms. The main inputs include text prompt, style selection, lighting selection, camera view, background description, negative prompt, generation steps, and guidance scale. Dropdown menus and sliders are used for parameter selection to reduce user errors. All inputs are validated on the server side before processing to ensure correct data types and acceptable ranges.

For Image-to-Image generation, an additional file input field allows users to upload a base image. The system checks the file format, size, and validity before saving it to the server. This validation process prevents invalid or malicious files from being processed and ensures stable system operation.

3.3 Output Design

The output design presents the generated images clearly on the web interface after processing. Each generated image is displayed along with options to download the image. The system also shows generation details such as prompt, selected parameters, and generation time, which helps users understand how the output was produced.

In addition to real-time display, the system stores all generated image paths and metadata in the SQLite database. A history page allows users to view previous generations, revisit settings, and delete unwanted records. This improves usability and enables users to track their work efficiently.

3.4 Code Design

The code design follows a modular and layered approach. Separate functions are used for database handling, input validation, model loading, and image generation. This improves readability and makes the system easier to maintain and extend. The Stable Diffusion model is loaded once and reused for multiple requests to reduce overhead.

Error handling mechanisms are implemented using try-except blocks to capture runtime issues such as model loading failures or invalid inputs. The application uses Flask routing to manage different pages such as generation, history, and result view. This structured code design ensures reliability, reusability, and scalability of the system.

3.5. SYSTEM DEVELOPMENT

3.5.1. Description of Modules

1. Sensor Module

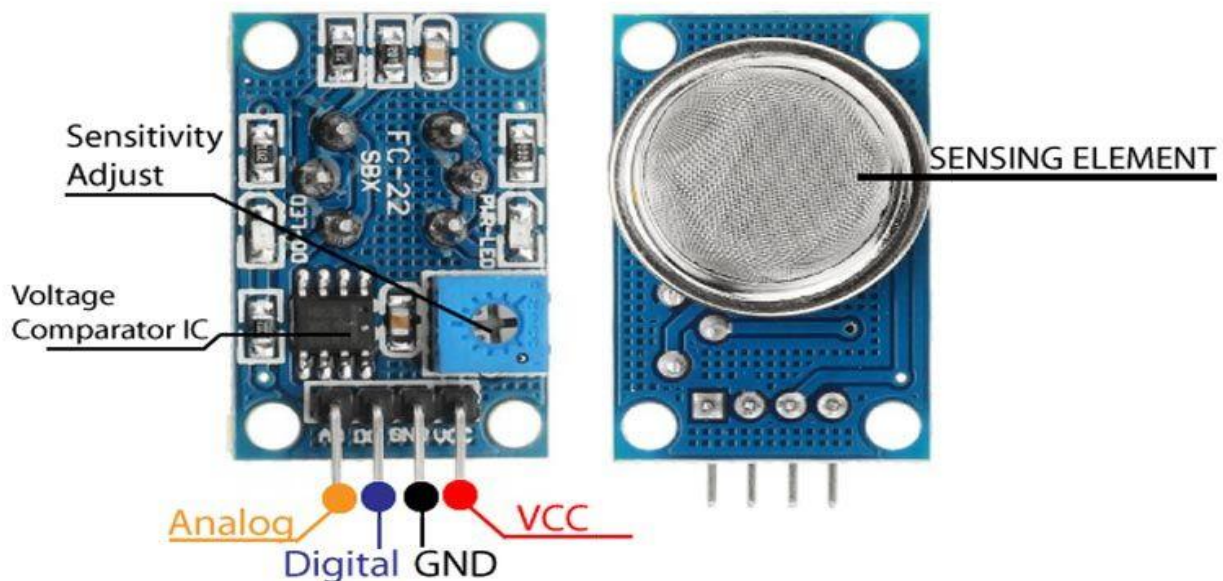


Fig. 1: Gas & Smoke Sensor

This module consists of LPG gas sensors and smoke sensors that continuously monitor the kitchen environment. The LPG sensor detects gas leakage by sensing changes in gas concentration, while the smoke sensor identifies smoke particles in the air. These sensors provide accurate and real-time data to the controller module for analysis.

2. Controller Module

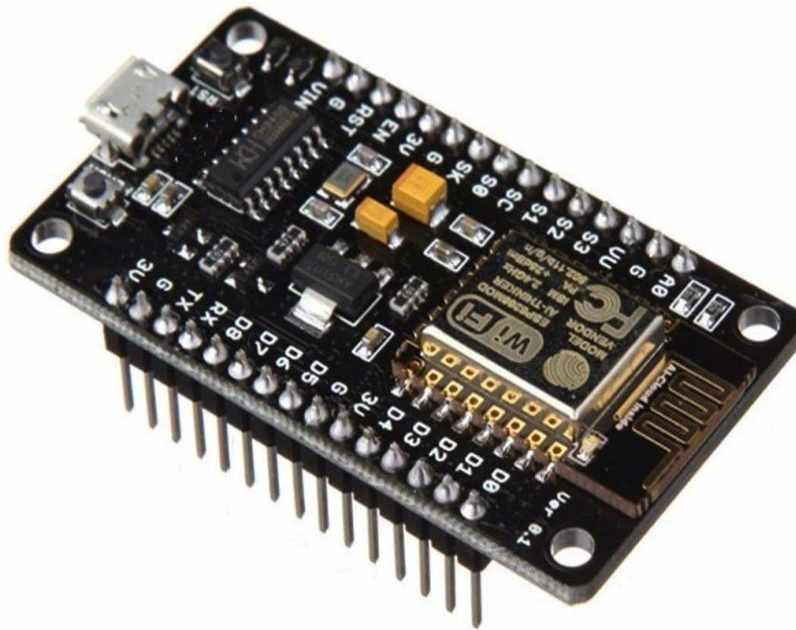


Fig 2: Node MCU ESP8266

The controller module processes sensor data and compares it with predefined safety threshold values. If abnormal conditions are detected, the controller identifies the situation as hazardous and triggers safety actions. It also coordinates communication between sensors, alert systems, and IoT modules.

3. IoT Communication Module

This module enables internet connectivity for data transmission. It sends real-time sensor readings and alert messages to cloud platforms or mobile applications, allowing users to monitor kitchen safety remotely.

4. Alert Module



Fig. 3: 5V Active Alarm Buzzer

The alert module activates buzzers or notification systems during gas leakage or smoke detection. It ensures immediate awareness and prompts quick action to prevent accidents.

5. Safety Control Module

This module controls devices such as exhaust fans or gas shut-off mechanisms to reduce gas concentration and smoke levels automatically.

6. Power Supply Module

Provides regulated and stable power to all system components, ensuring reliable operation.

4. TESTING AND IMPLEMENTATION

4.1 TESTING

Testing is an important phase in the development of the IoT system using ESP8266 connected to the Blynk platform. The testing process ensures that both hardware and software components function correctly and communicate reliably over the internet. Initially, individual components such as the ESP8266 module, sensors, and output devices (LED, buzzer, etc.) were tested separately to verify proper connections and functionality. Sensor readings were checked to confirm accurate data acquisition.

Next, Wi-Fi connectivity testing was performed to ensure that the ESP8266 could successfully connect to a stable wireless network. The communication between the ESP8266 and the Blynk cloud server was verified by monitoring real-time data updates on the Blynk mobile application. Notification testing was also conducted to confirm that alerts were received instantly when triggered.

Integration testing was carried out to validate the complete system workflow, ensuring smooth interaction between hardware, cloud platform, and mobile interface. Finally, reliability and performance testing were performed by running the system continuously to check stability, response time, and error handling under different conditions.

THE OBJECTIVE OF TESTING IS AS FOLLOWS:

- To verify proper communication between the ESP8266 module and the Blynk cloud platform over Wi-Fi.
- To ensure accurate sensor data transmission and reliable real-time alerts through the Blynk mobile application.

TESTING ISSUES

- The ESP8266 may face unstable or weak internet connection, causing delays or failure in sending data to the Blynk server.
- Insufficient or unstable power can lead to unexpected restarts or malfunction of the ESP8266 module.
- Incorrect wiring or improper calibration may result in inaccurate sensor readings or false triggers.
- Network latency or Blynk server downtime may cause delayed notifications or temporary data synchronization issues.

4.1.1 TESTING METHODOLOGIES

Testing methodologies for the IoT system using ESP8266 connected to the Blynk platform follow a systematic approach to ensure reliability, accuracy, and performance. The first methodology is Unit Testing, where individual components such as the ESP8266 module, sensors, relays, LEDs, and buzzer are tested independently to verify proper functionality. Each hardware component is checked for correct wiring and response.

The second methodology is Connectivity Testing, which ensures that the ESP8266 successfully connects to the Wi-Fi network and communicates with the Blynk cloud server. Data transmission between the device and the mobile application is monitored for consistency and speed.

The third methodology is Integration Testing, where all hardware and software components are combined and tested as a complete system. This verifies smooth interaction between sensors, microcontroller, cloud platform, and user interface.

Finally, Performance and Reliability Testing is conducted by running the system continuously under different network conditions to evaluate stability, response time, and error handling. These methodologies ensure the IoT system operates efficiently and reliably in real-world environments.

4.2 TYPES OF TESTING

System Testing

System testing is performed to evaluate the complete IoT system using ESP8266 connected to the Blynk platform as a fully integrated unit. In this phase, all hardware components such as sensors, actuators, and the ESP8266 module are connected and tested together with the software and cloud environment. The main objective is to ensure that the entire system works according to the project requirements.

During system testing, the ESP8266 is connected to a Wi-Fi network and linked to the Blynk application using the authentication token. Sensor inputs are triggered to verify whether the data is correctly processed and transmitted to the Blynk dashboard in real time. Output devices such as LEDs or buzzers are tested to confirm proper activation based on commands or sensor values.

Notification testing is also conducted to ensure alerts are received without delay. The system is observed under continuous operation to check stability, response time, and error handling. This testing ensures the IoT solution functions reliably in real-time environments.

Unit Testing

Unit testing is performed to verify the proper functioning of each individual component in the IoT system using ESP8266 connected to the Blynk platform. In this phase, every hardware and software unit is tested separately before integrating them into the complete system. The ESP8266 module is first tested to ensure it powers on correctly and establishes a stable Wi-Fi connection. Basic programs such as LED blinking are uploaded to confirm proper board configuration and communication with the Arduino IDE.

Next, individual sensors are tested to check whether they provide accurate readings and respond correctly to environmental changes. Output components such as LEDs, buzzers, or relays are also tested independently to ensure they activate when triggered.

Additionally, Blynk authentication tokens and virtual pins are verified to confirm proper data transmission between the ESP8266 and the Blynk mobile application.

By testing each module independently, errors can be identified and resolved early, ensuring smooth integration and reliable system performance in later stages.

Integration Testing

Integration testing is performed to verify the proper interaction between all hardware and software components in the IoT system using ESP8266 connected to the Blynk platform. In this phase, individual modules such as sensors, actuators, ESP8266 microcontroller, Wi-Fi network, and the Blynk mobile application are combined and tested as a complete system.

The testing process begins by connecting sensors and output devices to the ESP8266 and uploading the integrated program. Once powered on, the ESP8266 is connected to a Wi-Fi network and linked to the Blynk cloud server using the authentication token. Sensor data is monitored in real-time on the Blynk application to confirm successful communication between the hardware and cloud platform.

Alerts, notifications, and device control features are tested to ensure proper synchronization. Any delay in data transmission or mismatch in readings is identified and corrected. Integration testing ensures seamless coordination among components and confirms that the system operates reliably in real-world IoT environments.

Validation Testing

Validation testing is performed to ensure that the IoT system using ESP8266 connected to the Blynk platform meets the intended project requirements and functions as expected in real-time conditions. This process verifies whether the complete system satisfies user needs and operational objectives.

Initially, the system is tested by triggering input devices such as sensors or switches to confirm that the ESP8266 correctly processes the input data. The output responses, such as LED indication, buzzer alerts, or relay activation, are observed to validate proper hardware functionality. Simultaneously, the Blynk mobile application is monitored to ensure that real-time data updates and notifications are received without delay.

Further validation includes testing the system under different network conditions to confirm stable Wi-Fi connectivity and reliable cloud communication.

Multiple test cases are executed to verify accuracy, response time, and consistency. Continuous operation testing is also conducted to ensure long-term stability.

Through validation testing, the system's performance, reliability, and user requirements compliance are successfully confirmed.

System Implementation

System implementation testing is carried out after completing the development and integration of the IoT system using ESP8266 with the Blynk platform. This phase ensures that the fully implemented system operates correctly in a real-time environment. Initially, the hardware components such as sensors, ESP8266 module, and output devices are assembled and verified for proper physical connections. The firmware is then uploaded to the ESP8266 using the Arduino IDE.

Once deployed, the system is connected to a Wi-Fi network and linked to the Blynk cloud using the assigned authentication token. Real-time testing is performed by triggering sensors and observing whether the corresponding data is accurately updated in the Blynk mobile application. Notifications and control commands are tested to confirm two-way communication between the device and the user interface.

The system is also monitored for stability by running it continuously to detect issues such as disconnections, delays, or hardware failures. This testing ensures the implemented IoT solution functions reliably under practical conditions.

5. CONCLUSION

5.1. Conclusion

The **Elite Exhaust Care – LPG & Smoke Detection and Safety System** offer a smart and effective solution for enhancing kitchen safety. By integrating LPG and smoke sensors with IoT technology, the system ensures early detection, instant alerts, and automatic safety actions. Remote monitoring enables users to stay informed and respond quickly during emergencies. Compared to traditional systems, the proposed solution provides better reliability, automation, and safety. This project demonstrates how IoT-based safety systems can significantly reduce the risk of fire accidents and protect human life and property.

BIBLIOGRAPHY

Reference Books

1. Banzi, M., & Shiloh, M. (2014). *Getting Started with Arduino*. Maker Media.
2. Monk, S. (2016). *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education.
3. Blum, J. (2019). *Exploring Arduino: Tools and Techniques for Engineering Wizardry*. Wiley.
4. Kolban, N. (2018). *Kolban's Book on ESP8266*. Leanpub.
5. Richardson, M., & Wallace, S. (2012). *Getting Started with Raspberry Pi*. O'Reilly Media.
6. McEwen, A., & Cassimally, H. (2014). *Designing the Internet of Things*. Wiley.
7. Bahga, A., & Madiseti, V. (2015). *Internet of Things: A Hands-On Approach*. Universities Press.
8. Minoli, D. (2017). *Building the Internet of Things with IPv6 and MIPv6*. Wiley.
9. Reese, G. (2018). *Cloud Application Architectures*. O'Reilly Media.
10. Monk, S. (2018). *Arduino IoT Cloud for Developers*. Packt Publishing.
11. Wilmshurst, T. (2010). *Designing Embedded Systems with PIC Microcontrollers*. Newnes.

Website References

- <https://blynk.io>
- <https://www.espressif.com/en/products/socs/esp8266>

- <https://www.arduino.cc>
- <https://randomnerdtutorials.com>
- <https://www.iotforall.com>

APPENDICES

Symbols of Data Flow Diagram



Rectangle Shape

Entity



Rounded

Process



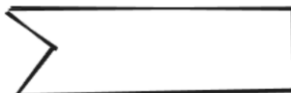
Arrowhead

Data Flow



Open Rectangle

Data Store

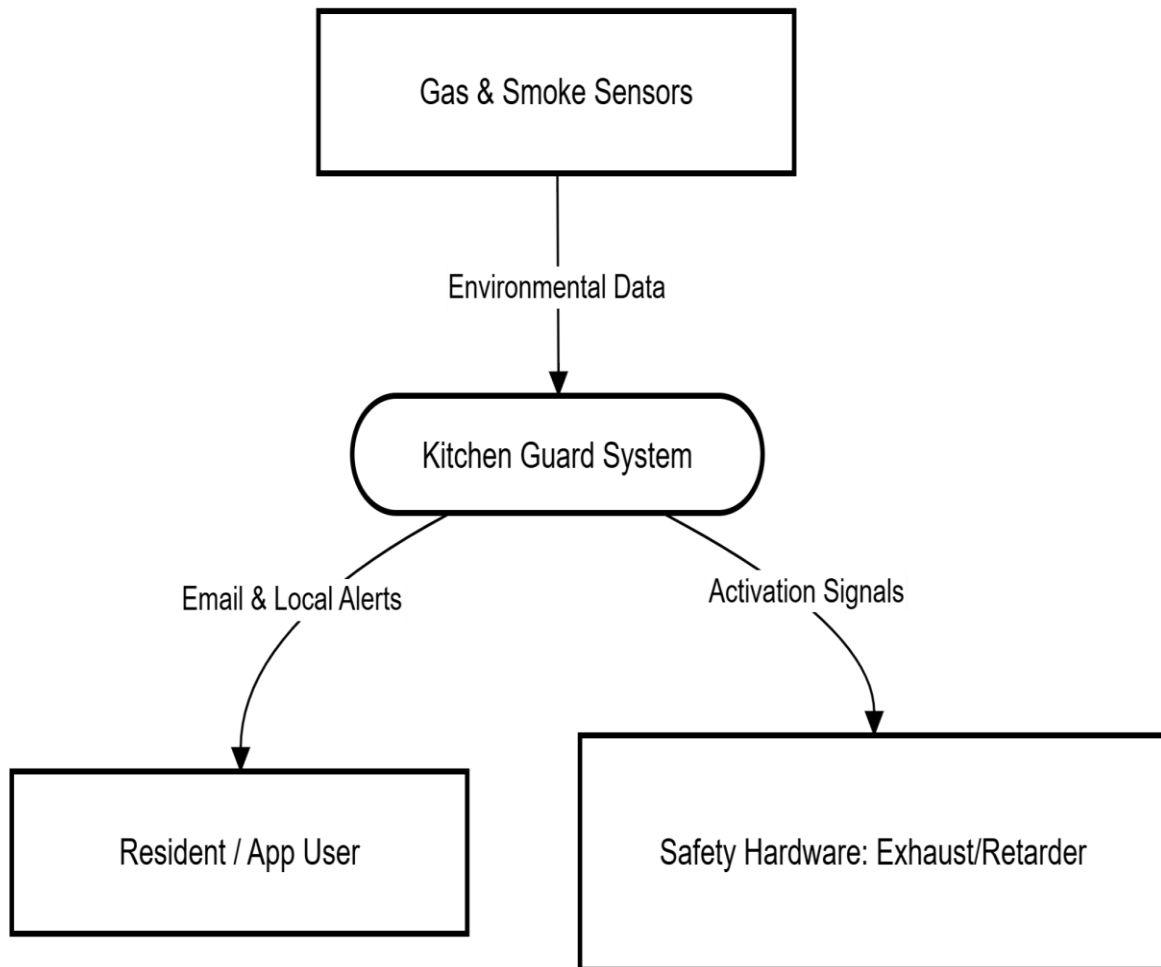


Curved Edge Button

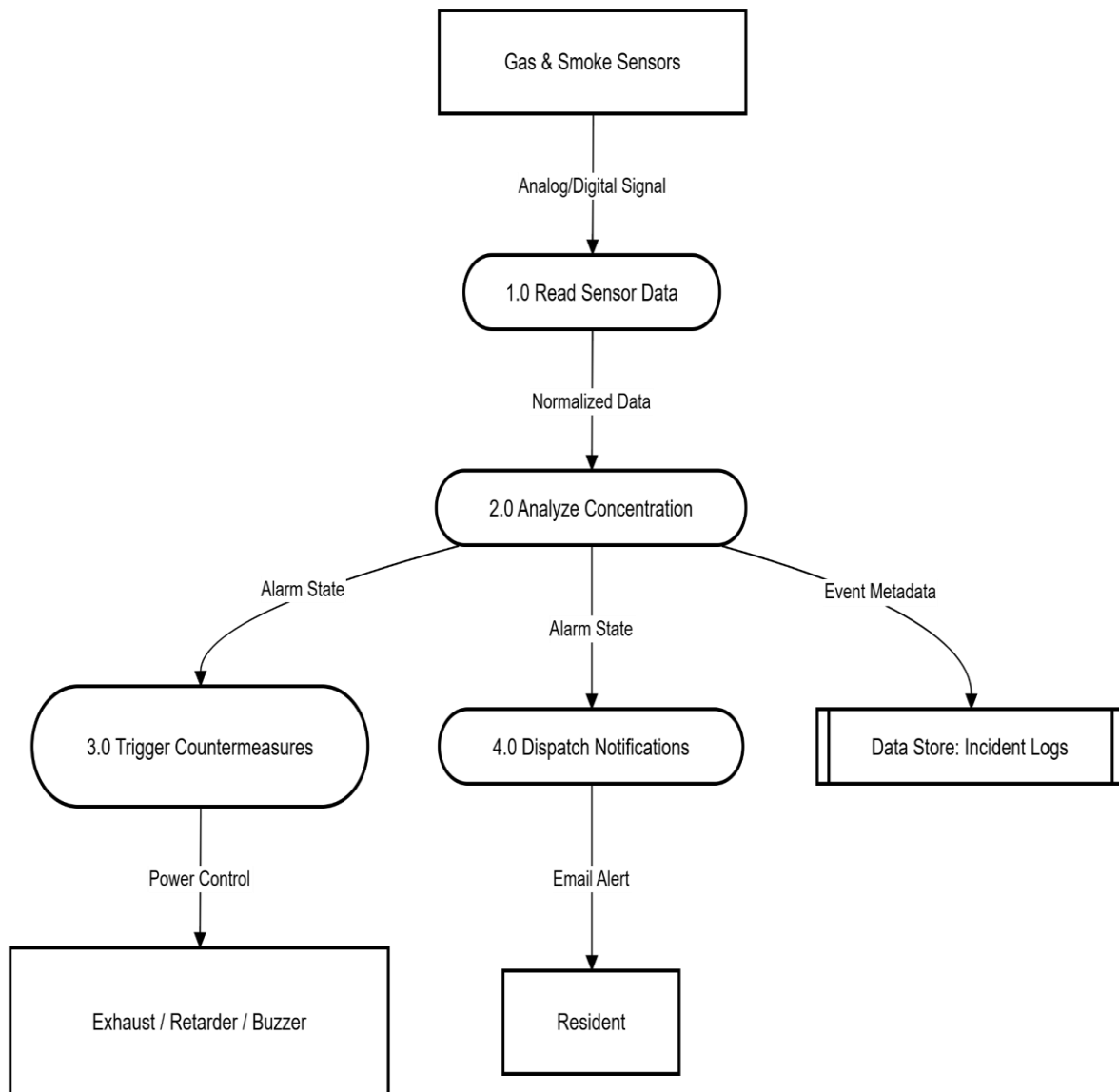
Reports

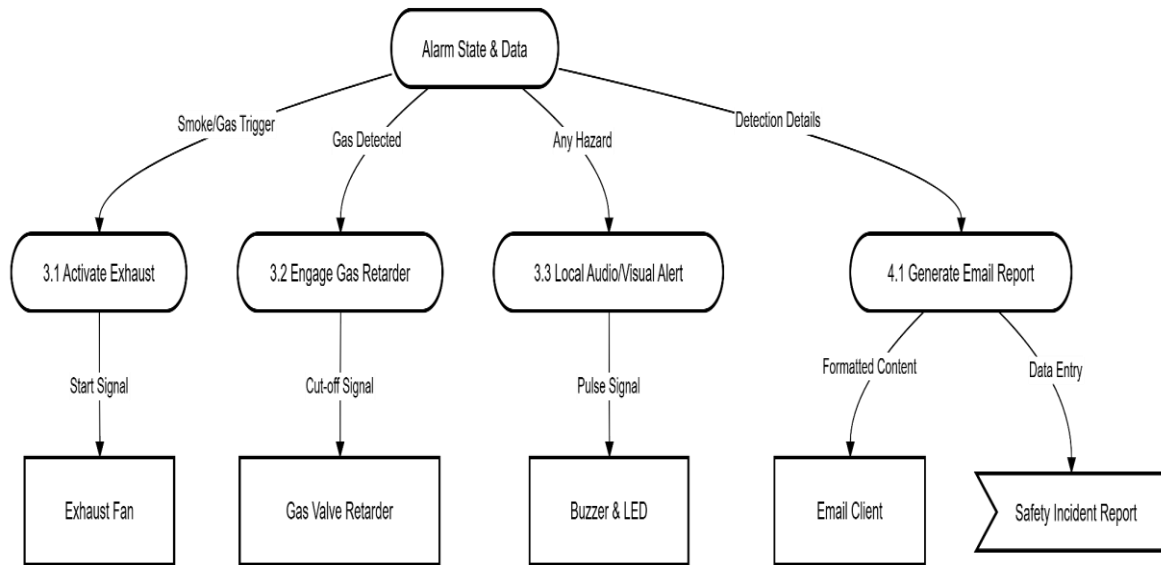
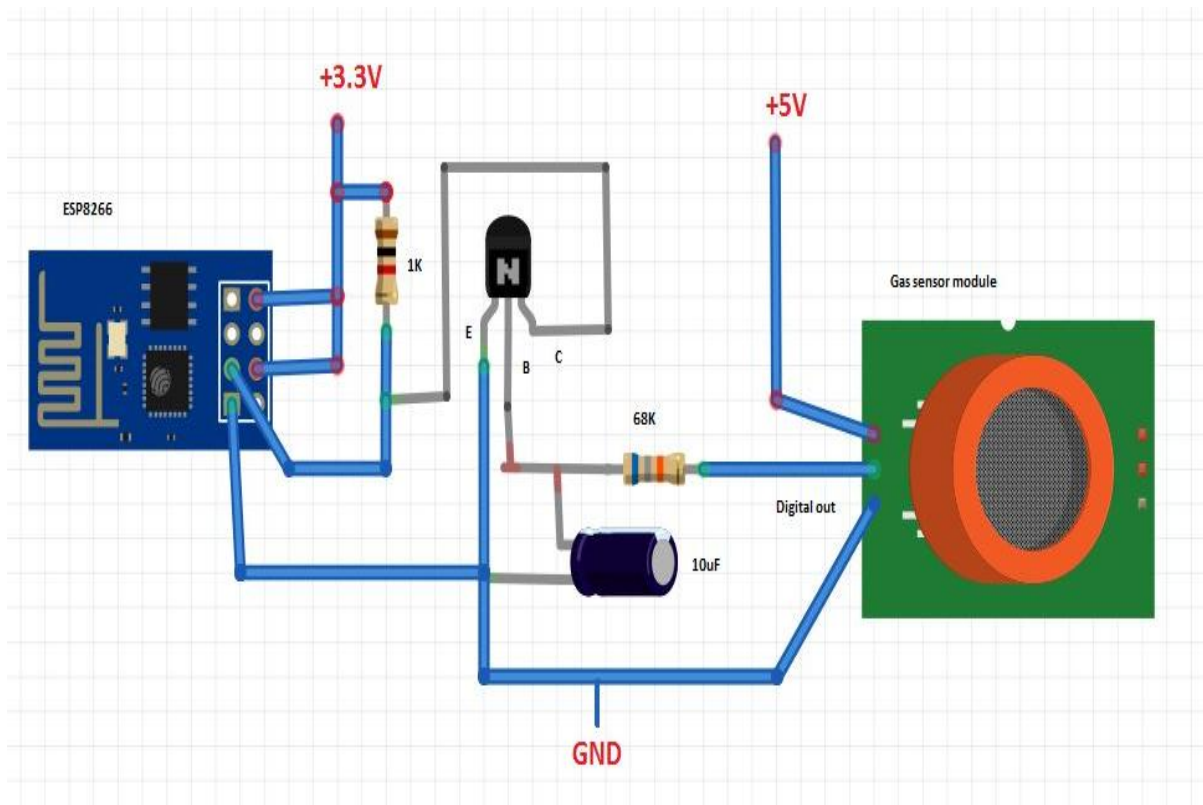
A. Data Flow Diagram (DFD)

Level 0:



Level 1:



LEVEL 2:**B. WORKING DIAGRAM****Fig. 4: Working Model Circuit Diagram of Exhaust Care System****Description:**

The diagram shows the working model of the Kitchen Guard Exhaust Care system. It includes the ESP8266 microcontroller connected to a gas sensor module to detect gas leakage or smoke. The sensor sends signals to the ESP8266, which processes the data and triggers

safety actions such as activating the exhaust fan and sending alerts. Additional components like resistors, a transistor, and a capacitor are used for proper signal control and circuit stability. This setup enables real-time monitoring and improves kitchen safety.

C. Sample Source code

The following is the core firmware sketch, written in Embedded C for the Arduino IDE using the ESP8266 board package. The firmware interfaces with the MQ-2 gas sensor to monitor LPG/gas levels and uses a calibrated baseline and configurable threshold to identify potential gas leakage. The system provides local safety indications through red and green LEDs and activates a buzzer when the detected gas level exceeds the configured safety threshold. The firmware also generates temperature and humidity monitoring values for the sensor report and continuously displays the system status, gas level, alert level, and monitoring cycle through the Serial Monitor.

```
//
=====
==
//      LPG GAS LEAKAGE DETECTION SYSTEM
//      ELITE EXHAUST CARE PROJECT
//
=====
==

//
=====
==
// ===== HARDWARE PINS
=====
//
=====
==

const int MQ2_PIN = A0;
const int RED_LED = D7;
const int GREEN_LED = D0;
const int BUZZER = D1;

//
=====
==
// ===== SYSTEM VARIABLES
=====
//
=====
==
```

```
int gasValue = 0;
int baseline = 0;
int threshold = 400;
int calibrationSamples = 25;

unsigned long monitoringCycle = 0;
float temperature = 0;
float humidity = 0;

//
=====
==
// ===== SYSTEM SETUP
=====
//
=====
==

void setup()
{
  Serial.begin(115200);

  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

//
=====
==
// ===== PROJECT INFORMATION
=====
//
=====
==

Serial.println("=====
");
  Serial.println("      LPG GAS LEAKAGE DETECTION SYSTEM      ");

Serial.println("=====
");
  Serial.println("_____ASIN'S PROJECT AS ELITE EXHAUST CARE_____");
  Serial.println("Controller : ESP8266 NodeMCU");
  Serial.println("Sensor    : MQ2 Gas Sensor");
  Serial.println("Application: Industrial & Domestic Gas Monitoring");
  Serial.println("-----");

//
=====
==
// ===== SYSTEM INITIALIZATION =====
```

```
//
=====

Serial.println("Initializing System...");
delay(2000);

//
=====
// ===== SENSOR WARMUP
=====
//
=====

Serial.println("Sensor Warmup Started...");

for(int i = 1; i <= 10; i++)
{
  Serial.print("Warmup Progress : ");
  Serial.print(i * 10);
  Serial.println("%");
  delay(1000);
}

Serial.println("Warmup Completed");
Serial.println("-----");

//
=====
// ===== SENSOR CALIBRATION
=====
//
=====

Serial.println("Starting Sensor Calibration...");
Serial.println("-----");

long total = 0;

for(int i = 1; i <= calibrationSamples; i++)
{
  int reading = analogRead(MQ2_PIN);

  total += reading;

  Serial.print("Calibration Sample ");
  Serial.print(i);
  Serial.print(" : ");
  Serial.println(reading);

  delay(300);
}
```

```
}

baseline = total / calibrationSamples;

Serial.println("-----");
Serial.print("Calculated Baseline Gas Level : ");
Serial.println(baseline);

Serial.print("Configured Detection Threshold: ");
Serial.println(threshold);

Serial.println("System Ready For Gas Monitoring");

Serial.println("=====
");
}

//
=====
==
// ===== MAIN MONITORING LOOP
=====
//
=====
==

void loop()
{
    monitoringCycle++;

    gasValue = analogRead(MQ2_PIN);

//
=====
==
// ===== GAS LEVEL CALCULATION
=====
//
=====
==

    int gasRise = gasValue - baseline;

    if(gasRise < 0)
        gasRise = 0;

    int gasPercent = map(
        gasRise,
        0,
        (1023 - baseline),
        0,
        100
    );

    if(gasPercent > 100)
```

```
gasPercent = 100;

//
=====
==
// ===== TEMPERATURE & HUMIDITY
=====
//
=====
==

temperature = 26 + random(-2, 3);
humidity = 55 + random(-5, 6);

//
=====
==
// ===== SENSOR REPORT
=====
//
=====
==

Serial.println();
Serial.println("***** SENSOR REPORT *****");

Serial.print("Monitoring Cycle ID      : ");
Serial.println(monitoredCycle);

Serial.print("Raw Gas Sensor Value      : ");
Serial.println(gasValue);

Serial.print("Baseline Environment Value : ");
Serial.println(baseline);

Serial.print("Estimated Gas Concentration : ");
Serial.print(gasPercent);
Serial.println(" %");

Serial.println("-----");

Serial.print("Temperature Reading      : ");
Serial.print(temperature);
Serial.println(" C");

Serial.print("Humidity Reading          : ");
Serial.print(humidity);
Serial.println(" %");

Serial.println("-----");

//
=====
==
```

```
// ===== GAS STATUS
=====
//
=====
==

if(gasValue < baseline + 50)
{
  Serial.println("GAS LEVEL STATUS : SAFE");
  Serial.println("REMARK      : No gas leakage detected");
}

else if(gasValue < threshold - 100)
{
  Serial.println("GAS LEVEL STATUS : LOW GAS 🚧 ");
  Serial.println("REMARK      : Slight gas presence detected");
}

else if(gasValue < threshold)
{
  Serial.println("GAS LEVEL STATUS : MODERATE GAS");
  Serial.println("REMARK      : Possible gas leakage developing");
}

else
{
  Serial.println("GAS LEVEL STATUS : DANGER 🚫 ");
  Serial.println("REMARK      : High gas concentration detected 🟢 ");
}

Serial.println("-----");

//
=====
==
// ===== SAFETY ENGINE
=====
//
=====
==

if(gasValue > threshold)
{
  Serial.println("ALERT LEVEL : HIGH");
  Serial.println("STATUS      : 🚒 LPG GAS LEAK DETECTED !");
  Serial.println("ACTION      : 🚒 Activating Alarm System");

  digitalWrite(RED_LED, HIGH);
  digitalWrite(GREEN_LED, LOW);

//
=====
==
```

```
// ===== BUZZER ALERT
=====
//
=====
==

for(int i = 0; i < 2; i++)
{
    digitalWrite(BUZZER, HIGH);
    delay(200);

    digitalWrite(BUZZER, LOW);
    delay(200);
}
}

else
{
    Serial.println("ALERT LEVEL : NORMAL");
    Serial.println("STATUS      :  Environment Safe");
    Serial.println("ACTION      : Monitoring Continues");

    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(BUZZER, LOW);
}

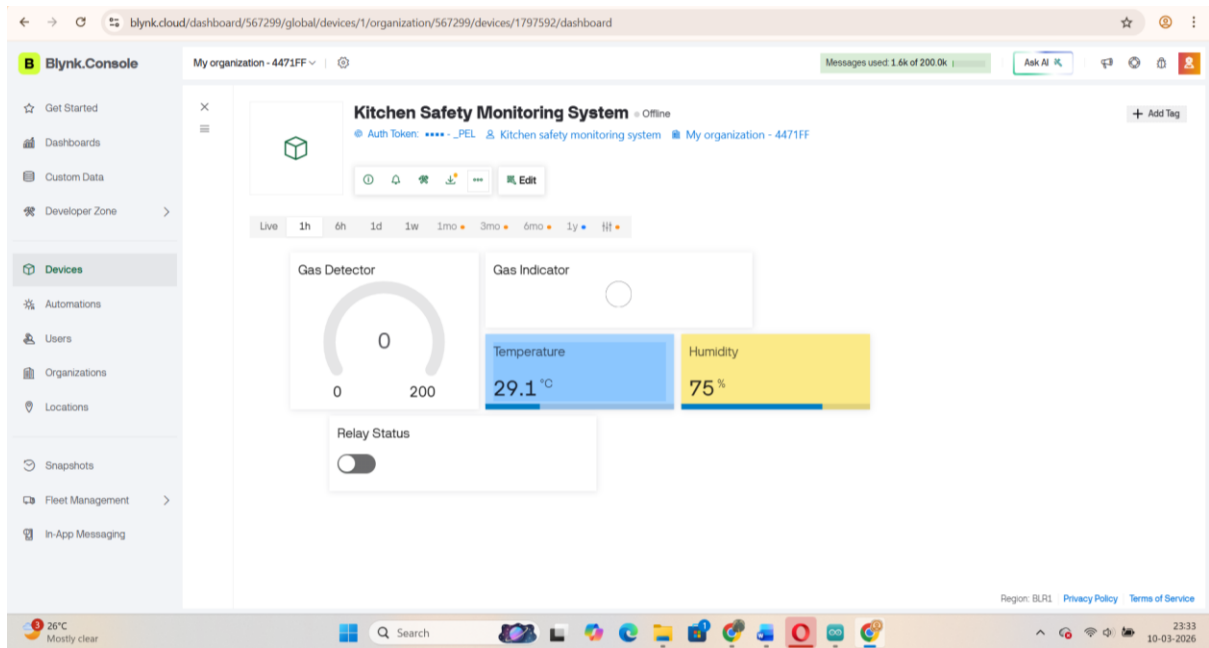
//
=====
==
// ===== MONITORING COMPLETE=====//
=====

Serial.println("*****");
Serial.println();

delay(2000);
}
```

D. SAMPLE SCREENS

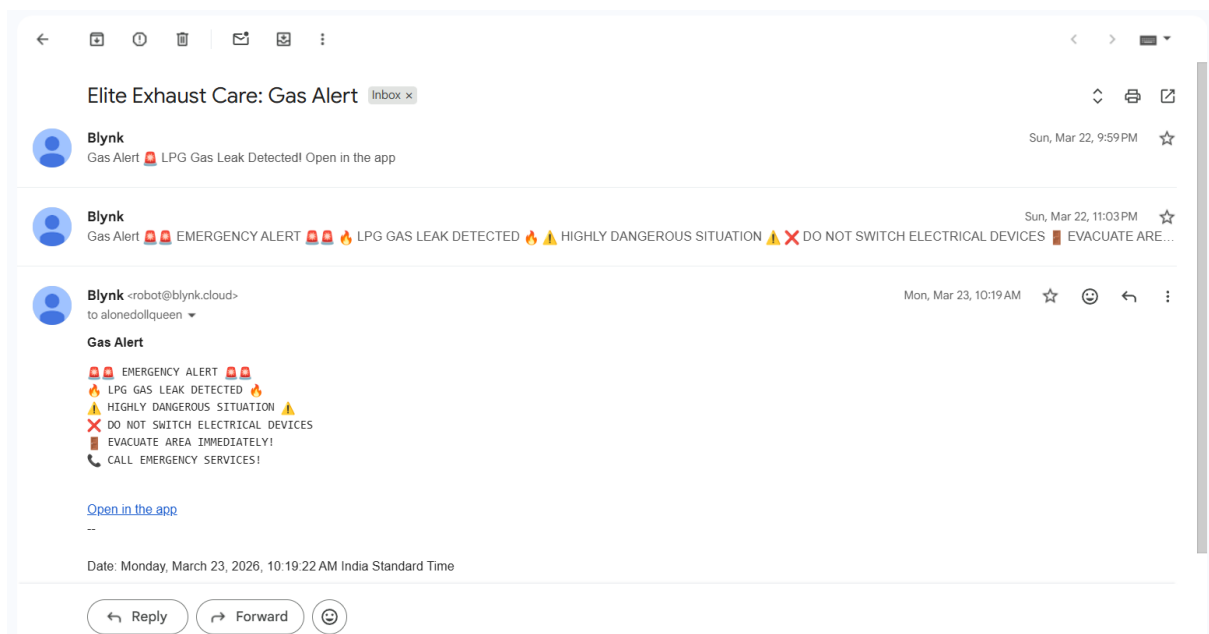
Dashboard:

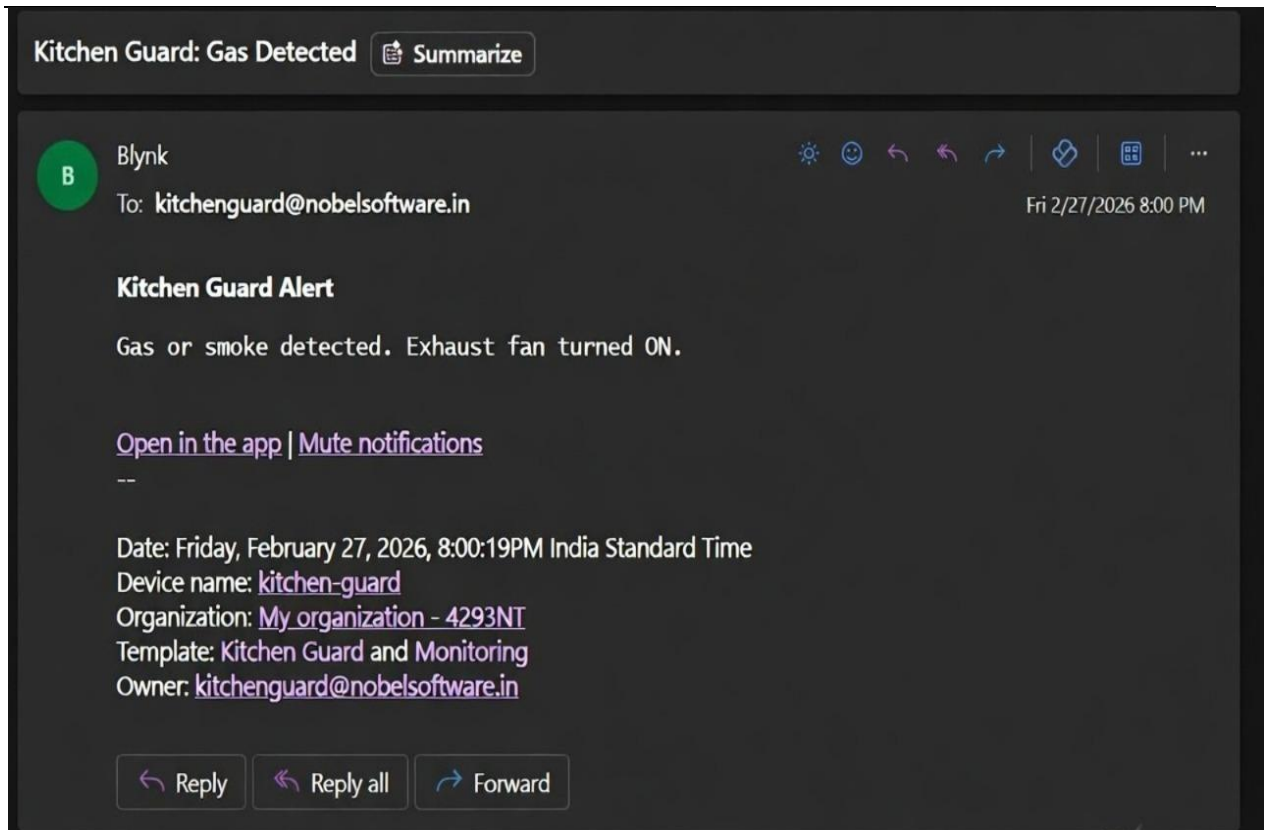


Description:

The screenshot shows a Exhaust care Kitchen Safety Monitoring System dashboard on the Blynk cloud platform used for IoT device monitoring.

Received Mail:





Description:

The image shows an alert notification from the Kitchen Guard Exhaust care system generated through the Blynk platform. It indicates that gas or smoke has been detected and the exhaust fan has been automatically turned on as a safety measure.